PART B

(PART B: TO BE COMPLETED BY STUDENTS)

Experiment 8

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded on the Blackboard or emailed to the concerned lab in charge faculties at the end of the practical in case the there is no Black board access available)

Roll.No: C-60	Name: Avdhut Madhavi
Class : SE-C	Batch: C2
Date of Experiment:	Date of Submission:
Grade:	

B.1: Read the given question carefully and write a PL/SQL statement and check the output.

Initial setup — create the MEMBER table

```
SET SERVEROUTPUT ON;

CREATE TABLE MEMBER (
   Member_ID NUMBER PRIMARY KEY,
   Name VARCHAR2(50),
   Phone VARCHAR2(15),
   Age NUMBER
);
```

Insert a sample record:

```
INSERT INTO MEMBER
VALUES (101, 'Rahul', '9876543210', 20);

COMMIT;
```

Q1. Create a BEFORE INSERT trigger

```
CREATE OR REPLACE TRIGGER member_before_insert
BEFORE INSERT ON MEMBER
FOR EACH ROW
BEGIN
   DBMS_OUTPUT.PUT_LINE('Before inserting member');
END;
/
```

Test it by inserting another member:

```
INSERT INTO MEMBER
VALUES (102, 'Priya', '9876543211', 22);
```

```
SQL> INSERT INTO MEMBER
  2  VALUES (102, 'Priya', '9876543211', 22);
Before inserting member

1 row created.
```

Q2. Create an AFTER INSERT trigger

```
CREATE OR REPLACE TRIGGER member_after_insert
AFTER INSERT ON MEMBER
FOR EACH ROW
BEGIN
   DBMS_OUTPUT.PUT_LINE('Member inserted successfully');
END;
/

INSERT INTO MEMBER
VALUES (103, 'Amit', '9876543212', 24);
```

```
SQL> INSERT INTO MEMBER
  2  VALUES (103, 'Amit', '9876543212', 24);
Before inserting member
Member inserted successfully

1 row created.
```

Q3. Create an INSERT trigger

```
CREATE OR REPLACE TRIGGER member_insert
AFTER INSERT ON MEMBER
FOR EACH ROW
BEGIN
   DBMS_OUTPUT.PUT_LINE('New member inserted');
END;
/

INSERT INTO MEMBER
VALUES (104, 'Neha', '9876543213', 21);
```

```
SQL> INSERT INTO MEMBER
  2  VALUES (104, 'Neha', '9876543213', 21);
Before inserting member
New member inserted
Member inserted successfully
```

Q4. Create an UPDATE trigger

```
CREATE OR REPLACE TRIGGER member_update
AFTER UPDATE ON MEMBER
FOR EACH ROW
BEGIN
```

```
   DBMS_OUTPUT.PUT_LINE('Member record updated');
END;
/

UPDATE MEMBER
SET AGE = 21
WHERE MEMBER_ID = 101;
```

```
SQL> UPDATE MEMBER
  2  SET AGE = 21
  3  WHERE MEMBER_ID = 101;
Member record updated

1 row updated.
```

Q5. Create a DELETE trigger

```
CREATE OR REPLACE TRIGGER member_delete
AFTER DELETE ON MEMBER
FOR EACH ROW
BEGIN
   DBMS_OUTPUT.PUT_LINE('Member record deleted');
END;
/

DELETE FROM MEMBER
WHERE MEMBER_ID = 101;
```

```
SQL> DELETE FROM MEMBER
  2  WHERE MEMBER_ID = 101;
Member record deleted

1 row deleted.
```

Q6. Display old and new age using :OLD and :NEW

```
CREATE OR REPLACE TRIGGER member_update_values
AFTER UPDATE ON MEMBER
FOR EACH ROW
BEGIN
   DBMS_OUTPUT.PUT_LINE('Old Age: ' || :OLD.AGE);
   DBMS_OUTPUT.PUT_LINE('New Age: ' || :NEW.AGE);
END;
/

UPDATE MEMBER
SET AGE = 23
WHERE MEMBER_ID = 102;
```

```
SQL> UPDATE MEMBER
  2  SET AGE = 23
  3  WHERE MEMBER_ID = 102;
Old Age: 22
New Age: 23
Member record updated

1 row updated.
```

Q7. Display the deleted member ID using :OLD

```
CREATE OR REPLACE TRIGGER member_delete_value
AFTER DELETE ON MEMBER
FOR EACH ROW
BEGIN
   DBMS_OUTPUT.PUT_LINE(
      'Deleted Member ID: ' || :OLD.MEMBER_ID
   );
END;
/

DELETE FROM MEMBER
WHERE MEMBER_ID = 103;
```

```
SQL> DELETE FROM MEMBER
  2  WHERE MEMBER_ID = 103;
Deleted Member ID: 103
Member record deleted

1 row deleted.
```

Q8. Display the new member ID using :NEW

```
CREATE OR REPLACE TRIGGER member_insert_value
AFTER INSERT ON MEMBER
FOR EACH ROW
BEGIN
   DBMS_OUTPUT.PUT_LINE(
      'New Member ID: ' || :NEW.MEMBER_ID
   );
END;
/

INSERT INTO MEMBER
VALUES (105, 'Sneha', '9876543214', 25);
```

```
SQL> INSERT INTO MEMBER
  2  VALUES (105, 'Sneha', '9876543214', 25);
Before inserting member
New Member ID: 105
New member inserted
Member inserted successfully

1 row created.
```

Q9. Create a trigger using UPDATE OF

```
CREATE OR REPLACE TRIGGER member_age_update
AFTER UPDATE OF AGE ON MEMBER
FOR EACH ROW
BEGIN
   DBMS_OUTPUT.PUT_LINE('Age has been updated');
END;
/

UPDATE MEMBER
SET AGE = 22
WHERE MEMBER_ID = 104;
```

```
SQL> UPDATE MEMBER
  2  SET AGE = 22
  3  WHERE MEMBER_ID = 104;
Age has been updated
Old Age: 21
New Age: 22
Member record updated

1 row updated.
```

B.2: Write an observation according to the query execution.

Observation

In this practical, different types of database triggers were created and executed using Oracle SQL. BEFORE and AFTER triggers were used to perform actions before and after INSERT operations. INSERT, UPDATE, and DELETE triggers were created to display messages when records were inserted, modified, or deleted. The :NEW and :OLD keywords were used to access new and previous column values, respectively. The UPDATE OF clause was used to execute a trigger when a particular column was updated.

Result: The creation and execution of database triggers were successfully performed, and their behavior was verified using SQL statements.

B.3: Write an answer of the following questions

1. What is a trigger?

A trigger is a database object that automatically executes when a specified event occurs on a table or view.

Triggers are commonly used with database operations such as INSERT, UPDATE, and DELETE. They can execute either before or after the specified operation.

2. When is a trigger executed?

A trigger is executed automatically when the database event specified in its definition occurs. The user does not need to execute the trigger manually.

Event	Meaning
INSERT	A new record is inserted.
UPDATE	An existing record is modified.
DELETE	An existing record is deleted.

3. What is the use of BEFORE?

BEFORE is a trigger timing specification that causes the trigger to execute before the specified database operation takes place.

It is useful when we need to validate or modify incoming data before it is stored in the database.

4. What is the use of AFTER?

AFTER is a trigger timing specification that causes the trigger to execute after the specified database operation takes place.

It is useful for performing follow-up actions, such as recording changes or maintaining audit information after a successful operation.

5. What is :NEW?

:NEW is a special trigger reference that represents the new values of a row's columns during an INSERT or UPDATE operation.

It is mainly used in row-level triggers to access the values being inserted or the updated values of a record.

6. What is :OLD?

:OLD is a special trigger reference that represents the previous values of a row's columns before an UPDATE or DELETE operation.

It is used to access the original values before they are changed or removed.

7. How do you disable a trigger?

A trigger can be disabled using the ALTER TRIGGER statement with the DISABLE keyword.

Disabling a trigger prevents it from executing when its triggering event occurs. The trigger definition remains stored in the database.

Syntax:

```
ALTER TRIGGER trigger_name DISABLE;
```

8. How do you enable a trigger?

A trigger can be enabled using the ALTER TRIGGER statement with the ENABLE keyword.

An enabled trigger automatically executes when its specified event occurs and its conditions are met.

Syntax:

ALTER TRIGGER trigger_name ENABLE;

9. How do you delete a trigger?
A trigger can be permanently removed from the database using the DROP TRIGGER statement.
Syntax:
DROP TRIGGER trigger_name;

10. What is a row-level trigger?
A row-level trigger is a trigger that executes separately for each row affected by an INSERT, UPDATE, or DELETE operation.
It is defined using the FOR EACH ROW clause.

Example:
```
CREATE OR REPLACE TRIGGER member_update
AFTER UPDATE ON MEMBER
FOR EACH ROW
BEGIN
    DBMS_OUTPUT.PUT_LINE('Member record updated');
END;
/
```